Agentic AI, Reinforcement Learning, and Quantum Control

Toward Hybrid RL + Agentic Systems for Physics Problems

Morten Hjorth-Jensen

April 23, 2026

Aim of These Notes

These slides develop three related ideas:

1. the conceptual link between reinforcement learning (RL) and agentic AI,
2. how this link appears naturally in quantum control,
3. how to build a hybrid RL + agentic system for physics problems.

The guiding perspective is that quantum control is inherently a sequential decision-making problem, while agentic AI provides a higher-level architecture for planning, tool use, and adaptive workflow management.

Reinforcement Learning

In reinforcement learning, an agent interacts with an environment over time:

$$s_t \rightarrow a_t \rightarrow r_t, s_{t+1}.$$

The objective is to maximize expected cumulative reward,

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t r_t \right],$$

where:

- s_t is the state,
- a_t is the action,
- r_t is the reward,
- $\pi(a|s)$ is the policy.

Agentic AI

Agentic AI refers to systems that:

- pursue explicit goals,
- make multi-step plans,
- use tools,
- adapt to feedback,
- and coordinate longer workflows.

It is broader than RL. A modern agentic system may combine:

- planning,
- memory,
- search,
- external simulation tools,
- and learned controllers.

The Link Between Them

The connection is:

RL is a mathematical training framework for agents, while agentic AI is a broader system architec

RL gives:

- a formal theory of sequential decision-making,
- value functions and policies,
- rewards and long-horizon optimization.

Agentic AI adds:

- planning over tasks,
- decomposition of objectives,
- tool orchestration,
- meta-level strategy.

Quantum Control Problem

A standard controlled Hamiltonian has the form

$$H(t) = H_d + \sum_k u_k(t) H_k,$$

where:

- H_d is the drift Hamiltonian,
- H_k are control Hamiltonians,
- $u_k(t)$ are external control amplitudes.

The goal is to choose $u_k(t)$ so that the evolution

$$i \frac{d}{dt} |\psi(t)\rangle = H(t) |\psi(t)\rangle$$

implements a desired state transfer or target gate.

Control Objective

For a target unitary U_{target}, one often maximizes a gate fidelity such as

$$F = \frac{1}{d^2} \left| \mathrm{Tr} \left(U_{\text{target}}^{\dagger} U(T) \right) \right|^2 .$$

Equivalently, one minimizes an infidelity

$$\epsilon = 1 - F .$$

A practical control objective may also include penalties:

$$\mathcal{J}[u] = 1 - F + \lambda_1 \mathcal{C}_{\text{amp}} + \lambda_2 \mathcal{C}_{\text{smooth}} + \lambda_3 \mathcal{C}_{\text{leak}} .$$

This already looks like a reward engineering problem.

Why Quantum Control Looks Like RL

Discretize time into N_t steps:

$$t_0, t_1, \ldots, t_{N_t}.$$

Then:

- state s_t: current wavefunction, propagator, or reduced feature representation,
- action a_t: control amplitude update $u_k(t)$,
- environment: Schrödinger evolution or open-system simulator,
- reward r_t: fidelity increment, final fidelity, or penalized objective.

So the control problem becomes:

choose actions over time to maximize final quantum performance.

MDP View of Quantum Control

The controlled quantum dynamics define a Markov decision process:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R).$$

One may choose:

$\mathcal{S}$: Bloch vectors, amplitudes, density matrix features, fidelities, leakage levels,

$\mathcal{A}$: pulse amplitudes, phases, frequencies, durations,

P : quantum dynamics + noise model,

R : reward based on gate fidelity, robustness, smoothness, runtime.

This is why RL has become natural in quantum control.

Classical Quantum-Control Methods vs RL

Traditional methods:

- GRAPE,
- Krotov,
- CRAB,
- Pontryagin-inspired optimal control.

These are powerful when:

- gradients are accessible,
- the model is known,
- the optimization landscape is manageable.

RL becomes attractive when:

- feedback is delayed,
- robustness matters,
- noise and uncertainty are significant,
- one wants adaptive or closed-loop behavior.

Why RL Alone Is Not the Whole Story

RL optimizes a policy inside a fixed environment and reward structure.
But realistic physics problems often require additional tasks:

- selecting the control model,
- choosing a simulation fidelity,
- changing the reward definition,
- diagnosing failure modes,
- comparing methods,
- adapting to experimental data.

These are naturally agentic tasks.

Agentic Layer Above Quantum Control

A higher-level agent can:

- choose whether to use GRAPE, RL, Bayesian optimization, or hybrid methods,
- call simulation tools,
- inspect fidelities, leakage, and robustness,
- refine pulse parametrizations,
- trigger retraining under new constraints,
- summarize and compare candidate controllers.

So RL can be one module inside a broader agentic architecture.

Two-Level Architecture

A useful conceptual split is:

Low level:

- a learned controller or RL policy that outputs pulse actions.

High level:

- an agentic planner that decides which tasks to run, which models to trust, and how to adapt the workflow.

In shorthand:

$$\text{Agentic planner} \longrightarrow \text{RL controller} \longrightarrow \text{quantum system.}$$

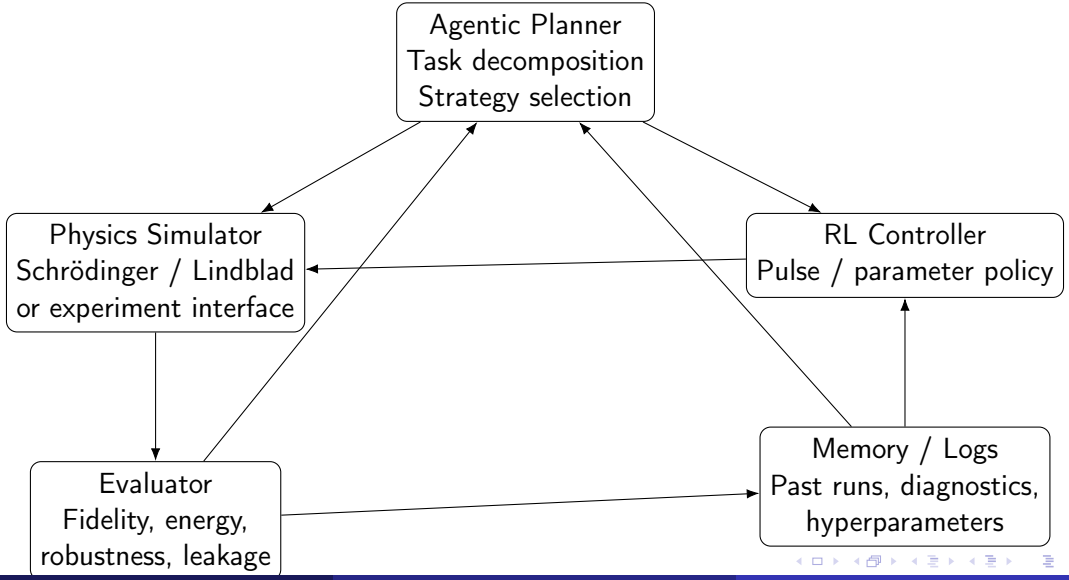
General Hybrid RL + Agentic System

A generic architecture for physics problems contains:

1. a **planner**,
2. one or more **physics simulators**,
3. an **RL controller**,
4. an **evaluator**,
5. a **memory / experiment log**,
6. a **policy for adaptation**.

This is useful whenever the problem is too complicated to solve with a single optimizer alone.

Architecture Diagram



What the Agentic Planner Actually Does

The planner can perform tasks like:

- define the target task,
- choose the state representation,
- choose the reward,
- choose the controller family,
- request more data,
- switch between coarse and fine simulations,
- compare candidate policies,
- terminate or continue search.

This is more like scientific workflow management than standard RL.

Example: Two-Qubit Gate Synthesis

Suppose we want to synthesize a target two-qubit gate $U_\star$ under the Hamiltonian

$$H(t) = H_d + u_x^{(1)}(t)X_1 + u_y^{(1)}(t)Y_1 + u_x^{(2)}(t)X_2 + u_y^{(2)}(t)Y_2.$$

The RL controller chooses the controls

$$a_t = \left(u_x^{(1)}(t), u_y^{(1)}(t), u_x^{(2)}(t), u_y^{(2)}(t)\right).$$

The agentic planner decides:

- whether to optimize a CZ or iSWAP gate,
- whether to include noise,
- whether to penalize leakage strongly,
- whether to use RL or switch to GRAPE for final polishing.

Reward Design in Hybrid Quantum Control

A typical RL reward might be

$$r_T = \alpha F_T - \beta L_T - \gamma P_T,$$

where:

- F_T: target fidelity,
- L_T: leakage penalty,
- P_T: pulse power or roughness penalty.

A more shaped reward could be

$$r_t = \Delta F_t - \lambda_1 \Delta L_t - \lambda_2 |a_t|^2.$$

The agentic layer can modify the coefficients $\alpha, \beta, \gamma, \lambda_i$ depending on the current training stage.

Example Agentic Strategy

A realistic multi-stage strategy:

1. use coarse simulation and cheap RL training to find rough pulses,
2. evaluate robustness to detuning and decoherence,
3. if good, switch to higher-fidelity simulation,
4. re-optimize with a refined reward,
5. optionally hand off to GRAPE for local polishing,
6. validate on noisy and perturbed models.

This is exactly the type of behavior that a pure RL loop does not naturally organize on its own.

Hybrid RL + Agentic Systems for Physics Problems

The same architecture applies beyond quantum control.
Examples:

- adaptive experiment design,
- quantum sensing,
- materials search,
- PDE-constrained optimization,
- active learning for phase diagrams,
- variational many-body wavefunction optimization.

The common structure is:

$$\text{sequential control or design} + \text{simulation / experiment} + \text{higher-level planning.}$$

Example: Quantum Sensing

In sensing, the RL controller may optimize:

- interrogation times,
- control pulses,
- measurement bases,
- adaptive feedback rules.

The agentic layer may:

- choose which parameter to estimate first,
- switch protocols depending on current uncertainty,
- decide when to collect more data,
- compare strategies using Fisher information or QFI.

Example: Variational Many-Body Physics

In many-body optimization, an RL component may tune:

- variational parameters,
- Monte Carlo update proposals,
- annealing schedules.

The agentic layer may:

- choose ansatz families,
- switch sampling strategies,
- inspect convergence and variance,
- refine the search over model classes.

Hierarchical Optimization View

One can formalize the hybrid system as a bilevel optimization problem.
Low level:

$$\pi_\phi^\star = \arg\max_{\pi_\phi} J(\pi_\phi; c),$$

where c denotes context or task settings chosen by the higher-level agent.
High level:

$$c^\star = \arg\max_{c} \mathcal{U}\big(\pi_\phi^\star(c), \mathcal{D}, \mathcal{M}\big),$$

where:

- $\mathcal{D}$ may be data or logs,
- $\mathcal{M}$ may be model choice or simulation settings.

So the agentic system optimizes the context in which RL itself is run.

Control, Estimation, and Meta-Decision

There are really three layers:

1. **dynamics:** Schrödinger / Lindblad / PDE evolution,
2. **control:** optimize actions for a fixed task,
3. **meta-decision:** choose the task framing, tools, and adaptation logic.

RL naturally occupies layer 2.
Agentic AI naturally occupies layer 3.

Implementation Sketch

A concrete implementation might use:

- simulator: QuTiP, custom NumPy/SciPy, or experiment API,
- RL backend: PPO, TRPO, SAC, or policy-gradient method,
- planner: LLM-based or rule-based orchestration layer,
- memory: database of runs, fidelities, hyperparameters,
- evaluator: scripts for robustness, leakage, spectra, or QFI.

The planner does not need to solve the physics itself. It coordinates tools that do.

Minimal Pseudocode

```
while budget_not_exhausted:
    task = planner.choose_task(memory)
    env  = build_environment(task)
    policy = train_or_update_rl(env, memory)
    result = evaluate(policy, env)
    memory.store(task, policy, result)
    planner.update(memory)
```

The key point is that the planner is not merely an optimizer; it is a controller of the overall scientific workflow.

Why This Hybrid Approach Is Attractive

Advantages:

- combines long-horizon control with scientific workflow reasoning,
- handles changing objectives and constraints,
- can adapt to uncertainty and noise,
- supports tool use and model selection,
- is well suited to realistic physics pipelines.

Limitations and Cautions

Challenges:

- reward misspecification,
- simulation cost,
- instability of RL training,
- hard credit assignment,
- planner errors or overconfident meta-decisions,
- difficulty of formal guarantees.

So the architecture is powerful, but it must be used carefully and benchmarked against simpler baselines.

Take-Home Summary

The main points are:

- RL and agentic AI are linked because both concern sequential goal-directed action.
- Quantum control is naturally an RL problem once time is discretized and controls become actions.
- Realistic physics workflows often need more than RL alone; they need planning, tool use, and adaptation.
- A hybrid RL + agentic system separates:

$$\text{high-level planning} \quad \text{from} \quad \text{low-level control.}$$

Final Perspective

For quantum control and related physics problems, the most promising picture is not:

$$\text{RL alone}$$

but rather

$$\boxed{\text{Agentic scientific workflow} + \text{RL controller} + \text{physics simulator/experiment}}$$

This is the natural architecture when one wants systems that do not merely optimize pulses, but reason about how to run the full scientific task.